

Tecnológico de Monterrey

Maestría en Inteligencia Artificial Aplicada

TC5061 · Operaciones de Aprendizaje Automático

Proyecto Integrador MiBici

Etapa 2: MLflow Project y tracking avanzado

Manuel Alejandro Cardona Esquivel

Matrícula: A01795954

Equipo 32 — Entrega individual autorizada

Profesor: Dr. Iván Reyes Amezcua

25 de octubre de 2026

Índice

Resumen 3
1. Alcance de la entrega 3
2. Proyecto reproducible 4
2.1 Servidor de tracking con base de datos 4
2.2 Archivo MLproject 4
2.3 Ejecución con mlflow run y reproducción desde un clon limpio 5
3. Experimentación y artefactos 6
3.1 Particiones temporales y métricas 6
3.2 Convención de tracking 6
3.3 Búsquedas de hiperparámetros con nested runs 7
3.4 Registro de artefactos 9
4. Comparación de experimentos y selección del modelo 11
4.1 P1: predicción de demanda 12
4.2 P2: clasificación de riesgo 13
4.3 P3: segmentación de estaciones 14
4.4 Síntesis de la selección 15
5. Model Registry y ciclo de vida 16
5.1 Modelos registrados 16
5.2 Esquema de ciclo de vida: aliases 17
5.3 Linaje y verificación de carga 18
6. Flujo de trabajo con Git y GitHub 21
7. Hallazgos de reproducibilidad y limitaciones 22
8. Conclusiones 22
Referencias 23

Resumen

Esta segunda etapa del proyecto integrador lleva el uso de MLflow a un nivel profesional sobre los datos abiertos de MiBici. El repositorio se empaquetó como un MLflow Project con tres puntos de entrada (`ingest`, `train` y `register`), todas las corridas se lanzaron con `mlflow run` contra un servidor de tracking local respaldado por SQLite y cada corrida candidata registra el modelo con su firma, la versión de los datos, gráficas propias de la tarea y la configuración con que se ejecutó.

Para cada una de las tres preguntas se entrenaron al menos tres candidatas adicionales al baseline, con una búsqueda de hiperparámetros organizada como corridas anidadas (nested runs) y etiquetas que permiten filtrar por pregunta, familia de modelo y etapa de trabajo. La selección se hizo con VALIDATION (P1 y P2) o con métricas internas de TRAIN y estabilidad temporal (P3); TEST se reservó para la evaluación final.

Los modelos elegidos fueron **PoissonRegressor** para P1 (MAE de validación 18.1 % menor que el baseline), **HistGradientBoostingClassifier ajustado** para P2 (F1 de validación 12.8 % mayor) y **K-Means con k = 4** para P3 (Davies–Bouldin 10.6 % menor y ARI de 0.94 frente a la ventana histórica reducida). Los tres quedaron en el Model Registry con el alias `champion`, un `challenger` como siguiente mejor alternativa y linaje completo hasta la corrida, el commit y la versión de datos.

1. Alcance de la entrega

La entrega continúa directamente la Propuesta de proyecto: conserva el experimento `mibici-equipo32`, los datos versionados con `data_version = 7aa5e436d887` y las tres preguntas. **P1** estima cuántos viajes se iniciarán en cada estación al día siguiente (regresión estación-día); **P2** clasifica si una estación estará en riesgo de quedarse sin bicicletas o sin anclajes en las próximas dos horas (clasificación estación-hora) y **P3** agrupa las estaciones según su patrón de uso (agrupamiento).

El trabajo se organizó en tres partes: un proyecto reproducible (servidor de tracking, archivo MLproject y ejecución con `mlflow run`), la experimentación con registro sistemático de artefactos y la gestión del ciclo de vida en el Model Registry. El repositorio de esta versión es **https://github.com/alexcard33/TC5061-Proyecto-Equipo32-mcardona**. Todas las corridas

finales se lanzaron desde el commit `3fa495d` sin cambios locales (`git_sucio = False`) y el código final, con la selección de modelos, está en la rama principal en el commit `18a9fcc`.

2. Proyecto reproducible

2.1 Servidor de tracking con base de datos

El Model Registry requiere un almacenamiento respaldado por base de datos, por lo que el proyecto usa un servidor local de MLflow con SQLite como backend. El servidor guarda metadatos, métricas, parámetros, etiquetas y el registro de modelos en `mlflow.db`, y sirve los artefactos nuevos desde `mlartifacts/`. El comando está documentado en el README y se expone como `make mlflow-ui`:

```
mlflow server --backend-store-uri sqlite:///mlflow.db \
  --artifacts-destination ./mlartifacts --host 127.0.0.1 --port 5000
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

La URI del servidor y el nombre del experimento se centralizaron en `config.py` (`MLFLOW_TRACKING_URI` y `MLFLOW_EXPERIMENT`): la URI se lee de la variable de entorno del mismo nombre y, si no existe, usa `http://127.0.0.1:5000`. Los tres scripts de modelado dejaron de escribir directamente sobre el archivo SQLite y se conectan al servidor, de modo que `mlflow run` puede propagar la conexión a cada corrida. Los archivos `mlflow.db`, `mlruns/` y `mlartifacts/` están excluidos en `.gitignore`, y la revisión del historial confirmó que nunca se versionaron, igual que los datos de MiBici y los modelos serializados.

2.2 Archivo MLproject

El archivo `MLproject` de la raíz declara el entorno en `python_env.yaml`, que instala exactamente las versiones fijas de `requirements.txt` (Python 3.12.1, MLflow 3.17.1, scikit-learn 1.9.1, pandas 3.0.6), y define tres puntos de entrada parametrizados.

Tabla 1

Puntos de entrada del MLflow Project

Entry point	Comando	Parámetros
ingest	python -m mibici.dataset	Sin parámetros. Genera data/processed y data/manifest.json con data_version.
train	python -m mibici.modeling.train	question (p1, p2 o p3, obligatorio); model (modelo de la pregunta o search); params (hiperparámetros clave=valor); run_name.

Entry point	Comando	Parámetros
register	python -m mibici.modeling.register	question (all por defecto). Registra champion y challenger y verifica la carga.

Nota. Los valores por defecto de hiperparámetros, rutas, fechas de partición y semilla provienen de config.py; el entry point solo recibe lo que cambia.

El punto de entrada `train` invoca un despachador que importa el módulo de la pregunta y entrena un único modelo por corrida, de forma que cada `mlflow run` produce exactamente una corrida comparable. Los hiperparámetros se pasan como pares `clave=valor` separados por comas (por ejemplo, `learning_rate=0.05,max_iter=350`) y se combinan con los valores por defecto de `config.py`. Se descartó recibirlos en JSON porque en Windows `mlflow run` ejecuta el comando mediante `cmd`, que rompe las comillas del JSON; el formato elegido funciona igual en Windows, Linux y macOS.

2.3 Ejecución con mlflow run y reproducción desde un clon limpio

Todas las corridas de la entrega se lanzaron desde el MLflow Project, no desde notebooks; en MLflow quedan con la etiqueta de origen `PROJECT` y registran los parámetros del proyecto (`question`, `model`, `params`) junto con los hiperparámetros efectivos. Las corridas hijas de cada búsqueda se crean dentro del mismo proceso de su corrida padre. La reproducción desde un clon limpio sigue estos pasos, documentados en el README:

```
git clone https://github.com/alexcard33/TC5061-Proyecto-Equipo32-mcardona.git
cd TC5061-Proyecto-Equipo32-mcardona
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
make mlflow-ui                              # servidor (otra terminal)
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
OPTS="--experiment-name mibici-equipo32 --env-manager local"
mlflow run . -e ingest --env-manager local
git checkout -- data/manifest.json          # ver sección 7
mlflow run . -e train -P question=p1 -P model=poisson $OPTS
mlflow run . -e train -P question=p2 -P model=hgb \
  -P params=learning_rate=0.05,max_iter=350,max_leaf_nodes=63 \
  -P run_name=p2_candidato_hgb_tuneado $OPTS
mlflow run . -e train -P question=p3 -P model=kmeans $OPTS
mlflow run . -e register --env-manager local
```

La opción `--experiment-name mibici-equipo32` es necesaria porque los scripts registran en ese experimento y MLflow rechaza la corrida si el experimento del proyecto no coincide. Con `--env-manager local` se usa el entorno virtual ya instalado; sin esa opción, MLflow construye el entorno a partir de `python_env.yaml`. En Windows se define además `PYTHONUTF8=1`, porque la consola de MLflow imprime caracteres que la codificación cp1252 no admite.

3. Experimentación y artefactos

3.1 Particiones temporales y métricas

Se mantuvieron la misma partición temporal y las mismas métricas de la entrega anterior, condición necesaria para que la comparación contra el baseline sea válida. La selección nunca usó TEST.

Tabla 2

Particiones temporales y métricas por pregunta

Pregunta	TRAIN	Selección	TEST	Métrica principal	Complementarias
P1	ene–sep 2025 90,230 filas	VAL oct 2025 11,439 filas	nov–dic 2025 22,565 filas	MAE (menor)	RMSE, WAPE, R^2, sesgo, error del total diario
P2	ene–sep 2025 2,348,476 filas	VAL oct 2025 273,798 filas	nov–dic 2025 539,478 filas	F1 (mayor); desempate PR-AUC	Recall, precisión, recall por tipo de riesgo
P3	1 ene–19 oct 2025	Métricas internas de TRAIN y estabilidad	20 oct–31 dic 2025	Silhouette (mayor) y Davies–Bouldin (menor)	Calinski–Harabasz, ARI y NMI

Nota. En P2 el umbral de decisión de cada modelo se elige en VALIDATION maximizando F1. En P3 la estabilidad se mide con el ARI y el NMI entre la asignación de TRAIN y la de TEST, y frente a una ventana histórica reducida (1 abr–19 oct 2025).

3.2 Convención de tracking

Además de las etiquetas obligatorias de equipo (`equipo`), commit (`git_commit`, `git_sucio`) y versión de datos (`data_version`), cada corrida lleva tres etiquetas estándar que permiten filtrar y comparar en la interfaz, por ejemplo con `tags.question = "p2" and tags.stage_of_work = "candidate"`.

Tabla 3

Etiquetas estándar de las corridas

Etiqueta	Valores	Uso
question	p1, p2, p3	Separar las tres preguntas dentro del experimento único.
model_family	naive, glm_poisson, gradient_boosting, random_forest, linear, kmeans, hierarchical, gaussian_mixture	Comparar familias de algoritmos.
stage_of_work	baseline, search, candidate	Distinguir la referencia, las búsquedas y las candidatas.

3.3 Búsquedas de hiperparámetros con nested runs

Cada pregunta tiene una búsqueda en malla organizada como una corrida padre y una corrida hija por combinación. La corrida padre registra la malla, el criterio, la mejor combinación (`mejor_*`), el identificador de la corrida hija ganadora (`mejor_run_hijo`) y la tabla completa de resultados. Las hijas registran solo métricas de TRAIN y VALIDATION, para que TEST no influya en la elección. La mejor combinación de P1 y P2 se volvió a entrenar como candidata completa (`hgb_tuneado`), con todos sus artefactos.

Tabla 4

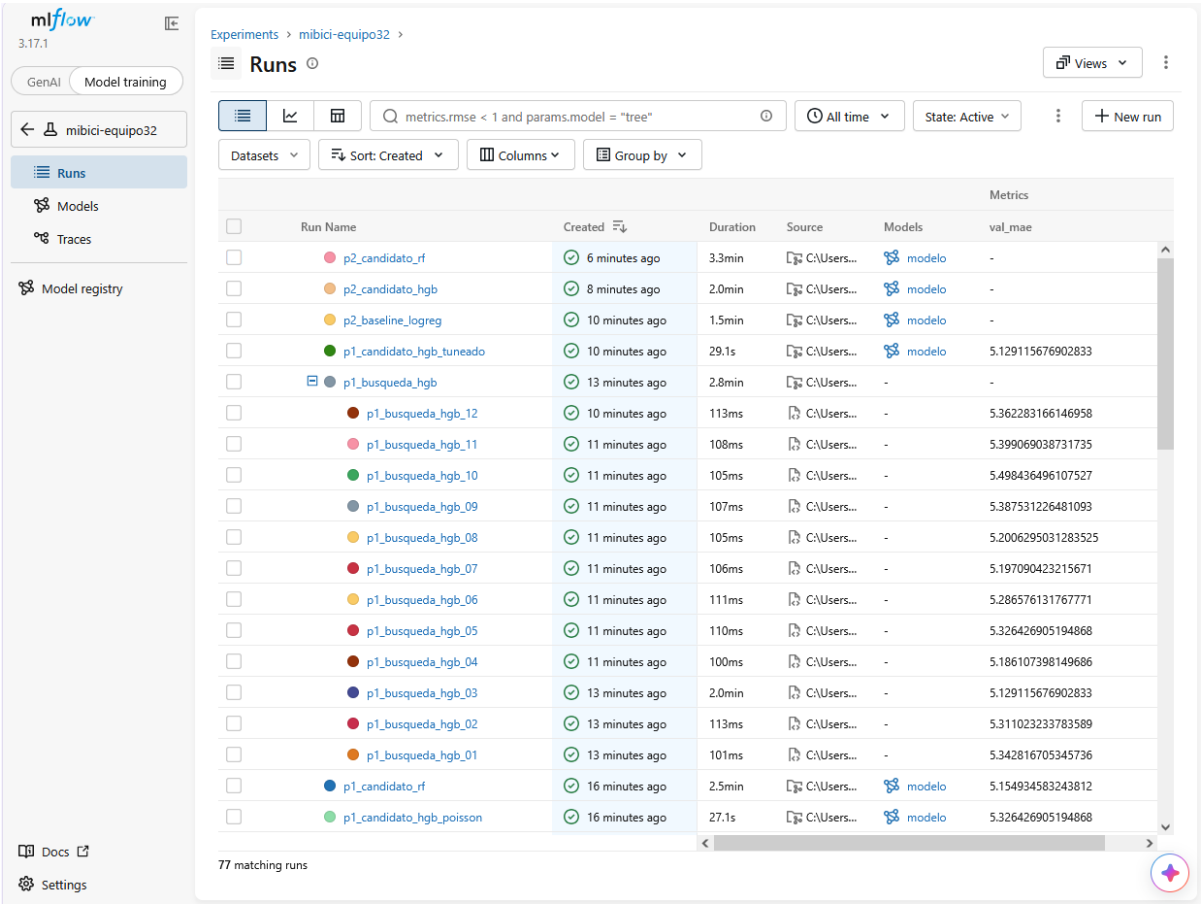
Búsquedas registradas como nested runs

Pregunta	Corrida padre	Malla	Hijas	Criterio	Mejor combinación
P1	p1_busqueda_hgb	learning_rate {0.03, 0.06, 0.1} × max_leaf_nodes {31, 63} × l2_regularization {1, 2}	12	Menor val_mae	lr=0.03, hojas=63, L2=1.0 (val_mae 5.1291)
P2	p2_busqueda_hgb	learning_rate {0.05, 0.1} × max_iter {200, 350} × max_leaf_nodes {31, 63}	8	Mayor val_f1; desempate val_pr_auc	lr=0.05, 350 iter, hojas=63 (val_f1 0.4681)
P3	p3_busqueda_kmeans	n_clusters {3, 4, 5, 6}	4	Mayor train_silhouette; desempate menor Davies–Bouldin	k=4 (silhouette 0.1864)

Las búsquedas muestran sensibilidades muy distintas. En P1 la malla tiene un efecto apreciable: el MAE de validación va de 5.1291 a 5.4984 y las combinaciones con 63 hojas y tasa de aprendizaje de 0.03 o 0.06 obtienen los mejores resultados, mientras que la tasa de 0.1 es la peor en todas las combinaciones de hojas y regularización. En P2 la malla es casi plana: las ocho combinaciones quedan entre 0.4656 y 0.4681 de F1, lo que indica que el desempeño está limitado por la información de las variables más que por la capacidad del modelo. En P3 la búsqueda de k es decisiva, como se analiza en la sección 4.3.

Figura 1

Corrida padre p1_busqueda_hgb con sus 12 corridas hijas en MLflow

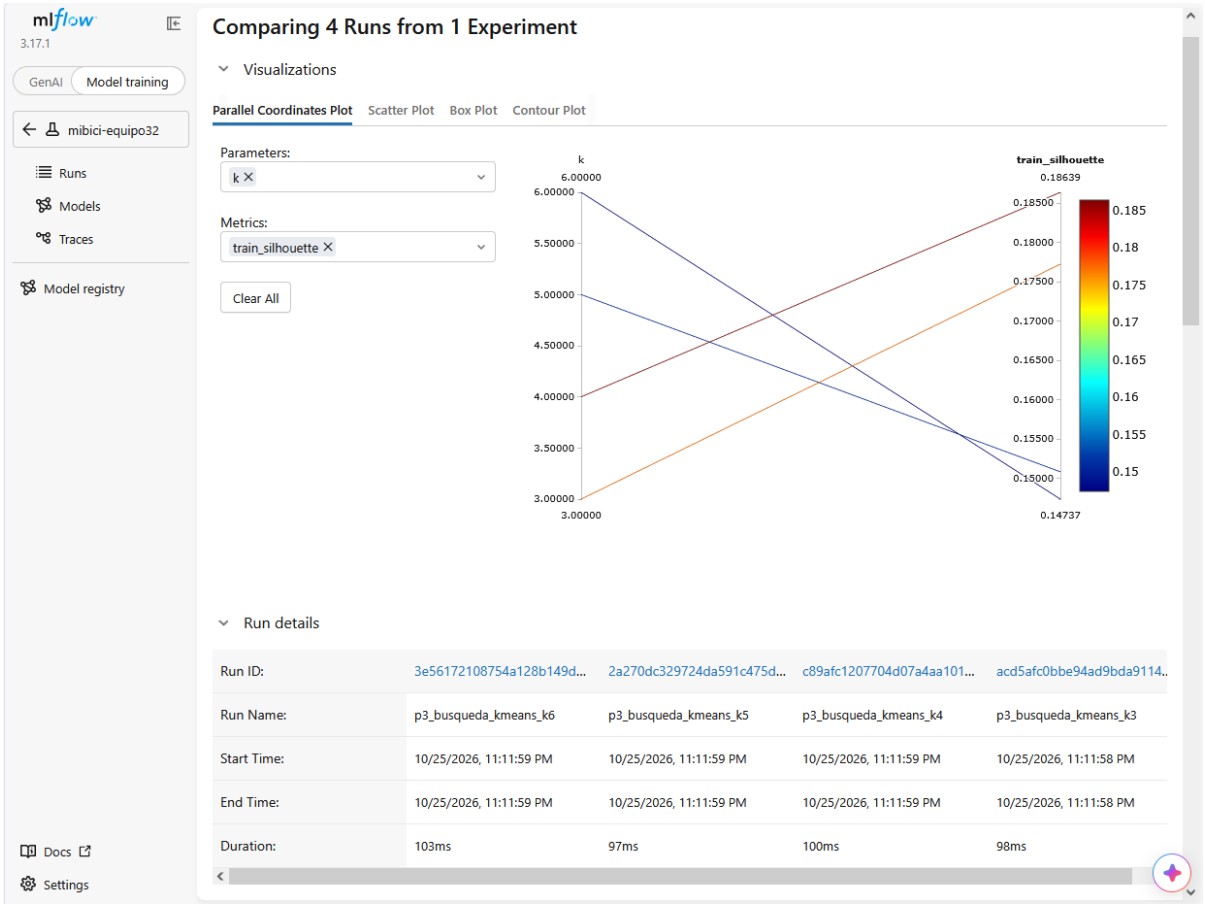


Run Name	Created	Duration	Source	Models	Metrics: val_mae
p2_candidato_rf	6 minutes ago	3.3min	C:\Users...	modelo	-
p2_candidato_hgb	8 minutes ago	2.0min	C:\Users...	modelo	-
p2_baseline_logreg	10 minutes ago	1.5min	C:\Users...	modelo	-
p1_candidato_hgb_tuneado	10 minutes ago	29.1s	C:\Users...	modelo	5.129115676902833
p1_busqueda_hgb	13 minutes ago	2.8min	C:\Users...	-	-
p1_busqueda_hgb_12	10 minutes ago	113ms	C:\Users...	-	5.362283166146958
p1_busqueda_hgb_11	11 minutes ago	108ms	C:\Users...	-	5.399069038731735
p1_busqueda_hgb_10	11 minutes ago	105ms	C:\Users...	-	5.498436496107527
p1_busqueda_hgb_09	11 minutes ago	107ms	C:\Users...	-	5.387531226481093
p1_busqueda_hgb_08	11 minutes ago	105ms	C:\Users...	-	5.2006295031283525
p1_busqueda_hgb_07	11 minutes ago	106ms	C:\Users...	-	5.197090423215671
p1_busqueda_hgb_06	11 minutes ago	111ms	C:\Users...	-	5.286576131767771
p1_busqueda_hgb_05	11 minutes ago	110ms	C:\Users...	-	5.326426905194868
p1_busqueda_hgb_04	11 minutes ago	100ms	C:\Users...	-	5.186107398149686
p1_busqueda_hgb_03	13 minutes ago	2.0min	C:\Users...	-	5.129115676902833
p1_busqueda_hgb_02	13 minutes ago	113ms	C:\Users...	-	5.311023233783589
p1_busqueda_hgb_01	13 minutes ago	101ms	C:\Users...	-	5.342816705345736
p1_candidato_rf	16 minutes ago	2.5min	C:\Users...	modelo	5.154934583243812
p1_candidato_hgb_poisson	16 minutes ago	27.1s	C:\Users...	modelo	5.326426905194868

Nota. La vista de corridas muestra la estructura padre-hijas de la búsqueda de P1 con el MAE de validación de cada combinación; la columna Models enlaza el modelo registrado por las corridas candidatas.

Figura 2

Comparación de las cuatro corridas hijas de p3_busqueda_kmeans



Nota. La gráfica de coordenadas paralelas relaciona el número de grupos k con la silhouette de TRAIN; k = 4 alcanza el valor más alto (0.1864) entre las combinaciones de la búsqueda.

3.4 Registro de artefactos

Cada corrida candidata guarda con MLflow los artefactos que respaldan su resultado, organizados en subcarpetas. El modelo se registra con `mlflow.sklearn.log_model`, con firma de entrada y salida (signature) y ejemplo de entrada (`input_example`); en MLflow 3 el modelo se almacena como un Logged Model vinculado a la corrida, por lo que aparece en la columna Models y en la pestaña Overview, no dentro del árbol de artefactos. Se verificó que los 27 modelos guardados en el experimento tienen firma.

Tabla 5

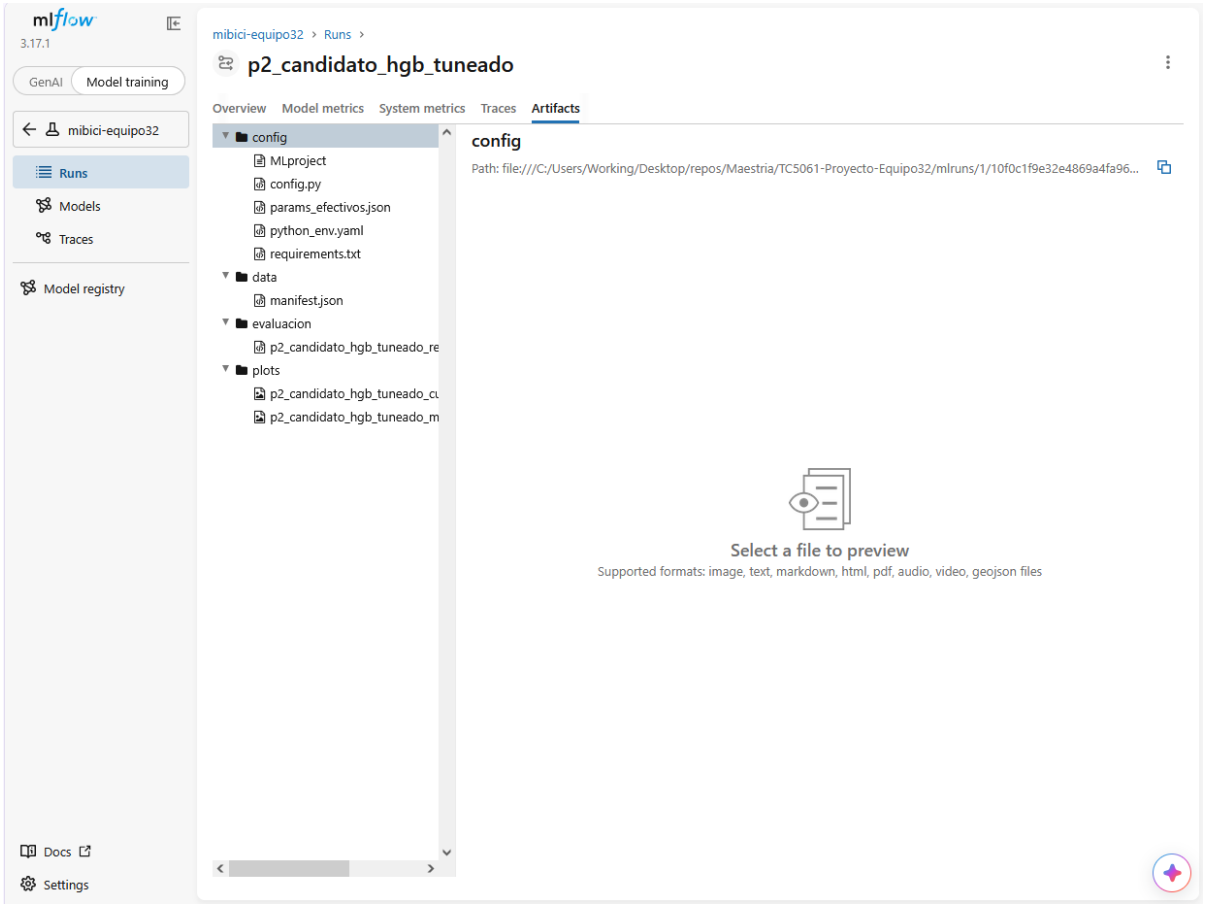
Organización de los artefactos de cada corrida candidata

Ubicación	Contenido
Logged Model	Modelo serializado (skops), firma, input_example, entorno (python_env.yaml, requirements.txt).
plots/	P1: real contra predicho y distribución de residuos, serie del total diario y MAE por día de la semana. P2: matriz de confusión y curva precisión-recall. P3: proyección PCA por perfil y mapa interactivo de grupos.
evaluacion/	Métricas en JSON, predicciones de TEST, importancia de variables, reporte de clasificación y tablas de perfiles.
data/	manifest.json de la ingesta (versión de datos). El conjunto de entrenamiento se registra además con mlflow.log_input (esquema, número de filas y digest).
config/	params_efectivos.json, config.py, MLproject, python_env.yaml y requirements.txt con que se ejecutó la corrida.

Nota. En P3 se guarda además model_bundle/, con el escalador y el mapeo de perfiles necesarios para usar el modelo. Ningún artefacto contiene datos personales: la ingesta elimina el pseudónimo, el género y el año de nacimiento de los usuarios antes de escribir los datos procesados.

Figura 3

Artefactos de la corrida candidata p2_candidato_hgb_tuneado



Nota. Se observan las subcarpetas config/, data/, evaluacion/ y plots/; el modelo con su firma está vinculado a la corrida como Logged Model.

4. Comparación de experimentos y selección del modelo

La selección de cada pregunta considera cuatro criterios: la mejora respecto al baseline en el conjunto de selección, la métrica ligada a la decisión de negocio, la estabilidad de los resultados entre TRAIN, VALIDATION y TEST, y el costo de entrenamiento e inferencia. Como indicador de costo de inferencia se usa el tamaño del modelo serializado y, como costo de entrenamiento, la duración de la corrida en el servidor de tracking, que incluye la preparación de datos.

4.1 P1: predicción de demanda

Tabla 6

Comparación de modelos de P1 contra el baseline

Modelo	Hiperparámetros clave	MAE TRAIN	MAE VAL	MAE TEST	Mejora VAL	Mejora TEST	MB	s
Baseline naive semanal	Rezago de 7 días	6.9659	6.2316	7.2195	Ref.	Ref.	—	12
PoissonRegressor	alpha=0.001, max_iter=1000	5.1950	5.1037	5.4098	+18.1 %	+25.1 %	0.18	21
HGB Poisson	lr=0.06, max_iter=400, hojas=31, L2=1.0	4.4370	5.3264	6.3395	+14.5 %	+12.2 %	2.9	27
Random Forest	n=200, prof.=16, hoja mín.=2, max_features=0.5	3.6492	5.1549	5.6861	+17.3 %	+21.2 %	205.5	150
HGB ajustado (búsqueda)	lr=0.03, max_iter=400, hojas=63, L2=1.0	4.4273	5.1291	5.9037	+17.7 %	+18.2 %	4.4	29

Nota. La mejora es la reducción porcentual del MAE frente al baseline. MB: tamaño del modelo serializado. s: duración de la corrida (incluye unos 6 s de preparación de datos). La fila resaltada es el modelo elegido.

Las cuatro candidatas superan al baseline en validación. **PoissonRegressor** obtiene el menor MAE de validación (5.1037, 18.1 % menos que el baseline), seguido muy de cerca por el HGB ajustado (5.1291) y el Random Forest (5.1549). Con diferencias tan pequeñas en validación, el criterio decisivo es la estabilidad: Poisson tiene un error casi idéntico en TRAIN, VALIDATION y TEST (5.20, 5.10 y 5.41), mientras que el Random Forest sobreajusta (MAE de 3.65 en TRAIN contra 5.15 en validación) y el HGB base se degrada 19 % de validación a TEST. Esta estabilidad se confirmó en la evaluación final: Poisson reduce el MAE de TEST en 25.1 % frente al baseline, la mayor mejora de todas las candidatas.

Para la decisión de negocio, la planeación diaria de rebalanceo, Poisson comete un error medio de 5.4 viajes por estación-día en TEST con un WAPE de 17.5 % (23.4 % en el baseline), y el error del total diario del sistema baja de 14.8 % a 9.1 %. Su sesgo es positivo (+1.36 viajes), lo que en operación se traduce en una ligera sobreestimación de la demanda, preferible a quedarse corto de bicicletas. En costo es la opción más eficiente: pesa 0.18 MB frente a 205.5 MB del Random Forest, su predicción es una combinación lineal y su forma (enlace logarítmico para conteos no negativos) es coherente con la variable objetivo.

Selección P1: PoissonRegressor (alpha = 0.001), corrida `p1_candidato_poisson_lineal`, como versión en producción. El HGB ajustado queda como challenger por ser la segunda mejor alternativa en validación.

4.2 P2: clasificación de riesgo

Tabla 7

Comparación de modelos de P2 contra el baseline

Modelo	Hiperparámetros clave	F1 TRAIN	F1 VAL	F1 TEST	PR-AUC VAL	PR-AUC TEST	Mejora F1 VAL	MB	s
Regresión logística	class_weight=balanced	0.4404	0.4151	0.3645	0.4146	0.3514	Ref.	0.04	91
HGB	lr=0.1, max_iter=200, hojas=31	0.4767	0.4680	0.4009	0.4836	0.3941	+12.7 %	1.5	121
Random Forest	n=200, prof.=12, hoja mín.=20, max_samples=0.3	0.4776	0.4595	0.3877	0.4755	0.3882	+10.7 %	48.1	200
HGB ajustado (búsqueda)	lr=0.05, max_iter=350, hojas=63	0.4818	0.4681	0.4014	0.4866	0.3995	+12.8 %	3.8	157

Nota. La mejora es el aumento porcentual del F1 de validación frente al baseline. Todos los modelos usan pesos balanceados para la clase minoritaria y un umbral de decisión elegido en VALIDATION. s incluye unos 12 s de preparación del panel estación-hora.

Las tres candidatas mejoran al baseline, y los dos modelos de gradient boosting quedan prácticamente empatados en F1 de validación (0.4681 el ajustado y 0.4680 el base). El criterio fijado antes de ver los resultados desempata por PR-AUC, que favorece al **HGB ajustado** (0.4866 frente a 0.4836); su mejora frente al baseline es de 12.8 % en F1 y de 17.4 % en PR-AUC. El Random Forest queda por debajo en ambas métricas y es el más costoso (48.2 MB y la corrida más larga).

La métrica se alinea con la decisión operativa: una alerta de riesgo dispara una visita de rebalanceo, de modo que importan tanto no omitir estaciones en riesgo (recall) como no saturar al equipo con falsas alarmas (precisión). En TEST, el modelo elegido detecta el 47.6 % de los eventos de riesgo con una precisión de 34.7 %, frente a 45.4 % y 30.5 % del baseline, y su recall es similar para ambos tipos de riesgo (44.7 % sin bicicletas y 50.1 % sin anclajes). La exactitud no se usa como criterio porque un clasificador que siempre predice «normal» alcanzaría 97.2 % de exactitud sin detectar ningún evento.

En estabilidad, todos los modelos pierden desempeño de validación a TEST en una proporción parecida (el F1 del modelo elegido baja de 0.468 a 0.401 y el del baseline de 0.415 a 0.365). La caída se explica en buena medida por el cambio de prevalencia: los eventos de riesgo son 3.94 % de las observaciones en octubre y 2.85 % en noviembre–diciembre, lo que reduce la precisión alcanzable con el mismo umbral. En costo, el modelo ajustado pesa 3.8 MB contra 1.5 MB del HGB base y su corrida tarda unos 35 s más; es una diferencia aceptable para un modelo que se reentrena con poca frecuencia, pero se documenta porque la ganancia en F1 es marginal.

Selección P2: HistGradientBoostingClassifier ajustado (learning_rate = 0.05, max_iter = 350, max_leaf_nodes = 63, umbral 0.8802), corrida `p2_candidato_hgb_tuneado`. El HGB base queda como challenger: ofrece un desempeño equivalente con menor costo y es la alternativa natural si en producción se prioriza la eficiencia.

4.3 P3: segmentación de estaciones

Tabla 8

Comparación de modelos de P3 contra el baseline

Modelo	Configuración	Silhouette TRAIN	D–B TRAIN	Mejora D–B	Silhouette TEST	ARI TRAIN–TEST	ARI histórico	MB
K-Means (baseline)	k=2, n_init=20	0.1977	1.7715	Ref.	—	—	—	—
K-Means	k=4, n_init=20	0.1864	1.5841	+10.6 %	0.1666	0.7927	0.9415	0.02
Agglomerative	k=4, enlace ward	0.1090	1.9998	−12.9 %	0.1368	0.3561	0.5234	0.03
Gaussian Mixture	k=4, covarianza full, n_init=5	0.1266	2.2180	−25.2 %	0.1146	0.7757	0.8746	0.05

Nota. D–B: índice de Davies–Bouldin (menor es mejor). La mejora es la reducción porcentual del índice frente al baseline. El modelo aglomerativo no tiene método predict, por lo que su asignación de TEST se obtiene reagrupando; el ARI no depende de la numeración de los grupos.

Tabla 9

Resultados de la búsqueda de k (corridas hijas de p3_busqueda_kmeans)

k	Silhouette TRAIN	Davies–Bouldin TRAIN	Calinski–Harabasz TRAIN	ARI histórico
3	0.1773	1.8178	85.94	0.6602
4	0.1864	1.5841	80.72	0.9415
5	0.1508	1.7201	71.69	0.9386

k	Silhouette TRAIN	Davies–Bouldin TRAIN	Calinski–Harabasz TRAIN	ARI histórico
6	0.1474	1.7807	64.36	0.4940

La calidad de un agrupamiento no se resume en una sola métrica. El baseline con k = 2 tiene la silhouette más alta (0.1977), pero dos grupos no distinguen los patrones operativos que interesan (estaciones de salida matutina, de llegada matutina, de fin de semana y de alta intensidad). Dentro de la búsqueda, **k = 4** obtiene la mejor silhouette, el menor Davies–Bouldin de todos los modelos (1.5841, 10.6 % menos que el baseline) y la mayor estabilidad frente a la ventana histórica reducida (ARI 0.9415); k = 3 y k = 6 resultan inestables (ARI de 0.66 y 0.49).

Entre las candidatas con cuatro grupos, K-Means domina en separación y estabilidad. El Gaussian Mixture conserva una estabilidad razonable (ARI TRAIN–TEST de 0.78), pero su Davies–Bouldin es 25.2 % peor que el del baseline y su entrenamiento es cinco veces más largo. El agrupamiento aglomerativo es inestable (ARI de 0.36 entre periodos) y no permite asignar estaciones nuevas sin reagrupar todo el conjunto, lo que lo descarta para producción. En TEST, usado solo después de la selección, K-Means mantiene a 91 % de las estaciones comparables en el mismo grupo, con un ARI de 0.79 y un NMI de 0.74. Su costo es mínimo: el modelo pesa 0.02 MB y la asignación es una distancia a cuatro centroides.

Selección P3: K-Means con k = 4 y n_init = 20, corrida `p3_kmeans_final`. El Gaussian Mixture queda como challenger por ser la otra candidata estable que admite predicción directa.

4.4 Síntesis de la selección

Tabla 10

Modelos seleccionados por pregunta

Pregunta	Champion	Challenger	Métrica de selección	Mejora vs. baseline
P1	PoissonRegressor p1_candidato_poisson_lineal	HGB ajustado	MAE VAL 5.1037	MAE 18.1 % menor en VAL y 25.1 % menor en TEST
P2	HGB ajustado p2_candidato_hgb_tuneado	HGB base	F1 VAL 0.4681; PR-AUC 0.4866	F1 12.8 % mayor en VAL y 10.1 % mayor en TEST
P3	K-Means k = 4 p3_kmeans_final	Gaussian Mixture	D–B TRAIN 1.5841; ARI histórico 0.9415	Davies–Bouldin 10.6 % menor en TRAIN

5. Model Registry y ciclo de vida

5.1 Modelos registrados

El entry point `register` lee la selección de `REGISTRO_MODELOS` en `config.py`, crea cada registered model con nombre fijo y descripción, registra como versión el modelo de la corrida elegida, copia a la versión las etiquetas de linaje, asigna los aliases y verifica que el modelo en producción se pueda cargar por nombre y alias. El comando es idempotente: si la corrida ya tiene una versión registrada, la reutiliza.

Tabla 11

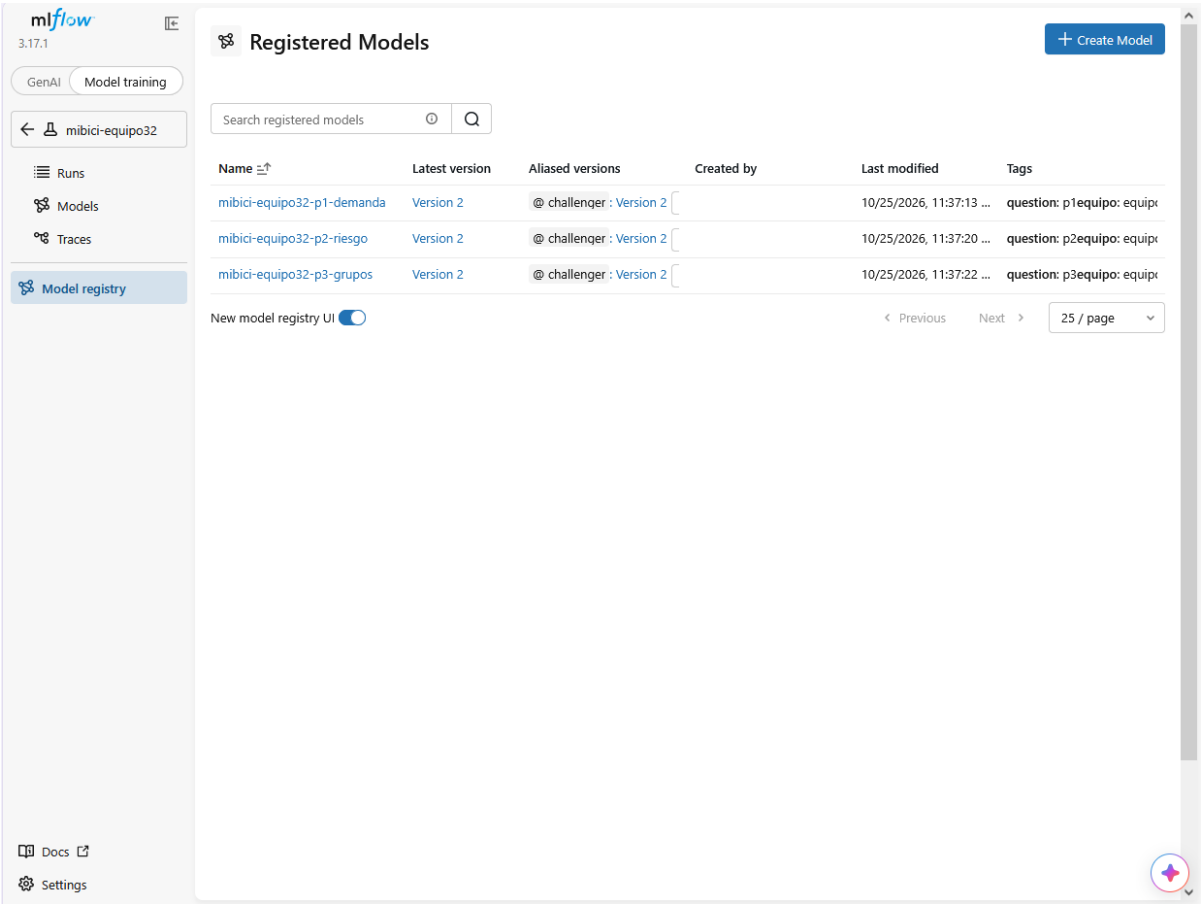
Versiones registradas y su linaje

Registered model	Ver.	Alias	Corrida de origen	Run ID
mibici-equipo32-p1-demanda	1	champion	p1_candidato_poisson_lineal	fb967ed88f20…
mibici-equipo32-p1-demanda	2	challenger	p1_candidato_hgb_tuneado	d3ec6fab58f6…
mibici-equipo32-p2-riesgo	1	champion	p2_candidato_hgb_tuneado	10f0c1f9e32e…
mibici-equipo32-p2-riesgo	2	challenger	p2_candidato_hgb	2f46693cdab5…
mibici-equipo32-p3-grupos	1	champion	p3_kmeans_final	3eecf6d03b50…
mibici-equipo32-p3-grupos	2	challenger	p3_candidato_gmm_k4	6914a57a0443…

Nota. Todas las versiones comparten experimento (mibici-equipo32), commit 3fa495d4dee084c711ab21a4fdeb7fd2b1a23609, git_sucio = False y data_version 7aa5e436d887, registrados como etiquetas de la versión junto con source_run_id, source_run_name, question, model_family y lifecycle_scheme = aliases. La versión de P2 guarda además umbral_decision.

Figura 4

Registered models de las tres preguntas en el Model Registry



Nota. Los tres modelos aparecen con su versión más reciente, los aliases asignados y las etiquetas del modelo registrado (question, equipo y lifecycle_scheme).

5.2 Esquema de ciclo de vida: aliases

Se eligió el esquema de aliases en lugar de las etapas clásicas (Staging, Production, Archived). Las etapas están obsoletas desde MLflow 2.9 y los aliases son el mecanismo vigente; además, ofrecen una ventaja operativa clara: el consumidor carga siempre `models:/<nombre>@champion` y promover, sustituir o revertir un modelo consiste en mover el alias a otra versión, sin cambiar código ni archivar versiones. Cada versión conserva su identidad y su linaje, de modo que el historial de qué estuvo en producción y por qué queda trazable.

El flujo previsto es el siguiente: una nueva versión se registra con el alias `challenger`, se evalúa contra la versión vigente con la misma partición y las mismas métricas y, solo si

mejora el criterio de su pregunta, recibe el alias `champion`. Revertir un cambio es tan simple como devolver el alias a la versión anterior. En esta entrega el challenger de cada pregunta es la siguiente mejor alternativa del análisis de la sección 4, lista para sustituir al modelo en producción si cambian las prioridades, por ejemplo en P2, donde el HGB base ofrece un desempeño equivalente con menor costo.

5.3 Linaje y verificación de carga

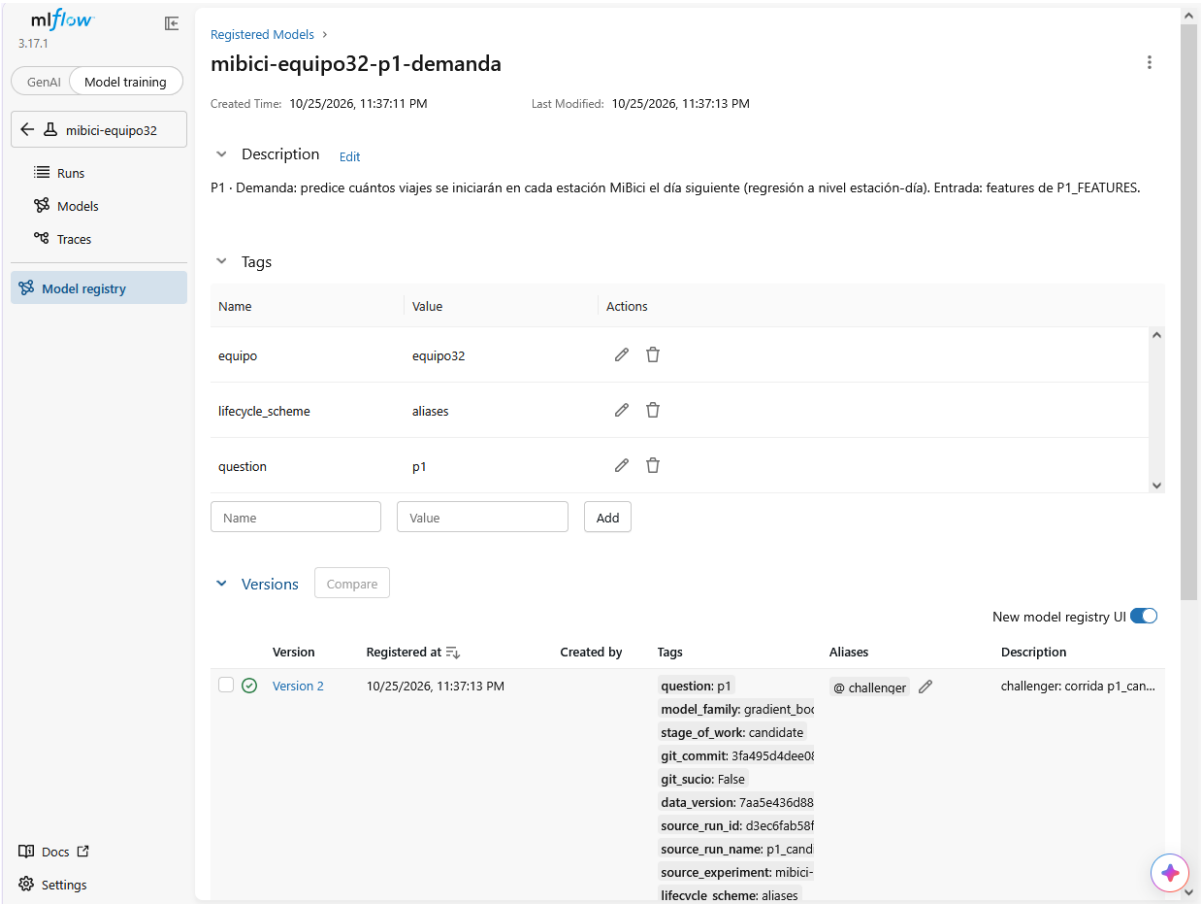
La verificación final cargó los tres modelos en producción por nombre y alias como modelos pyfunc, sin errores:

```
import mlflow
mlflow.pyfunc.load_model("models:/mibici-equipo32-p1-demanda@champion")
mlflow.pyfunc.load_model("models:/mibici-equipo32-p2-riesgo@champion")
mlflow.pyfunc.load_model("models:/mibici-equipo32-p3-grupos@champion")
```

Dos consideraciones de consumo quedaron documentadas en la descripción de cada registered model. En P2 la decisión se toma sobre `predict_proba` con el umbral elegido en validación (0.8802, guardado en la etiqueta `umbral_decision`), no con el umbral de 0.5 que usaría `predict`. En P3 el modelo espera las variables transformadas con log1p y estandarizadas con el escalador del artefacto `model_bundle/` de la corrida de origen.

Figura 5

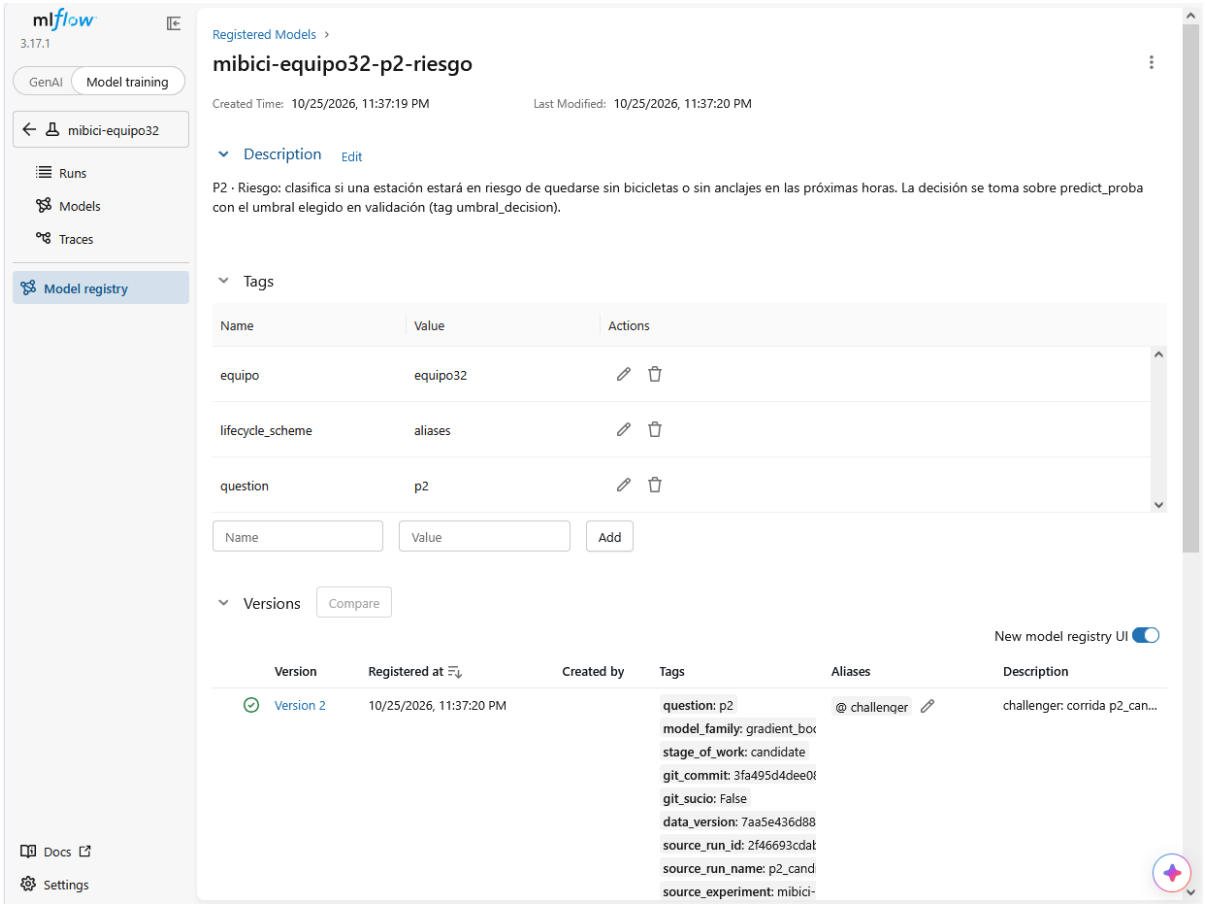
Registered model mibici-equipo32-p1-demanda



Nota. La página del modelo muestra su descripción, las etiquetas del modelo y la tabla de versiones con las etiquetas de linaje de cada versión: question, model_family, stage_of_work, git_commit, git_sucio, data_version, source_run_id, source_run_name y source_experiment.

Figura 6

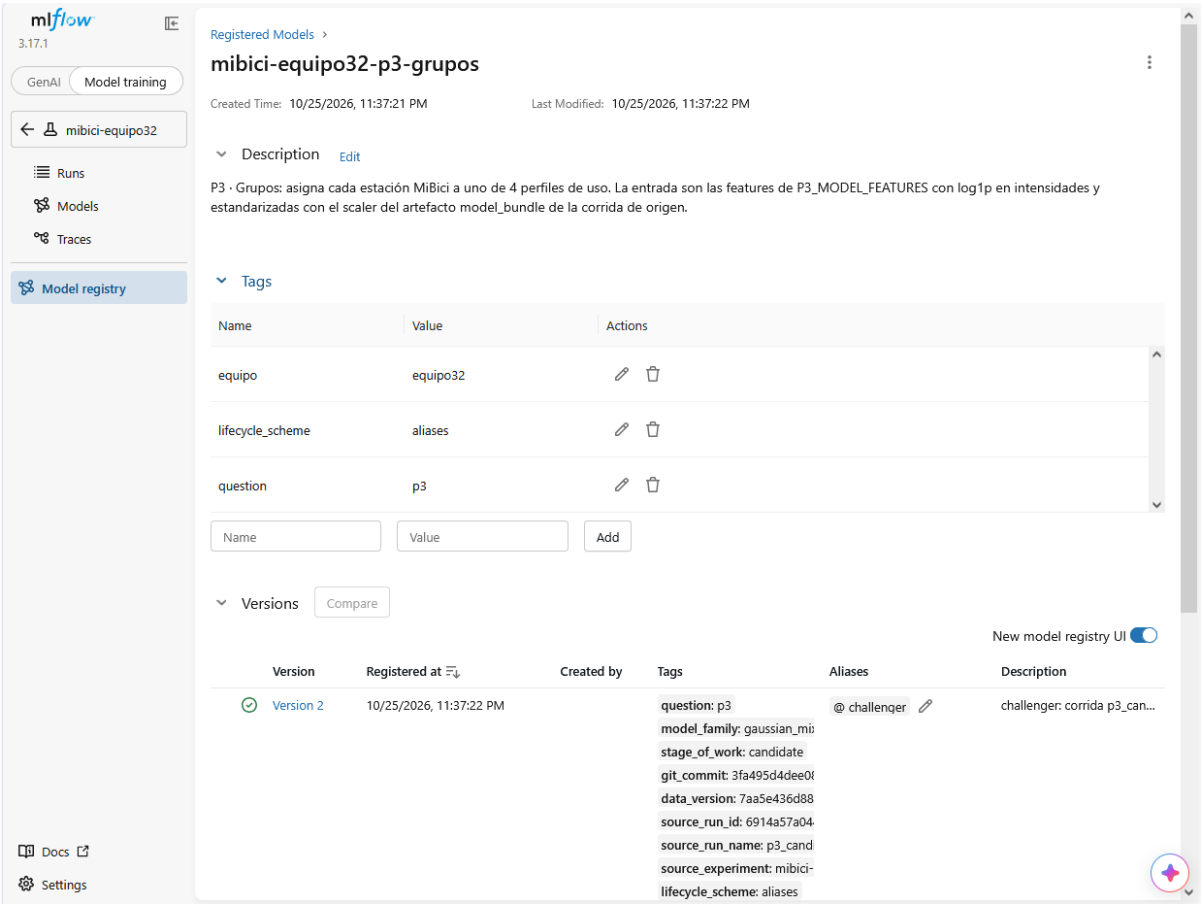
Registered model mibici-equipo32-p2-riesgo



Nota. La descripción indica cómo usar el umbral de decisión; las versiones conservan las mismas etiquetas de linaje hasta la corrida, el commit y la versión de datos.

Figura 7

Registered model mibici-equipo32-p3-grupos



Nota. La descripción documenta el preprocesamiento esperado por el modelo; las versiones registran su corrida de origen, el commit y la versión de datos.

6. Flujo de trabajo con Git y GitHub

Todo el trabajo de esta etapa entró a la rama principal mediante Pull Requests, uno por bloque funcional, de modo que el historial refleja la secuencia de construcción del proyecto.

Tabla 12

Integración de la etapa mediante Pull Requests

PR	Merge en main	Commit	Alcance
#1	c92487e	2133c58	Servidor de tracking con SQLite y URI centralizada en config.py.
#2	68292e4	935f8fa	MLflow Project: MLproject, python_env.yaml y entry points ingest y train parametrizados.
#3	2716887	3fa495d	Candidatas nuevas, búsquedas con nested runs, etiquetas estándar, artefactos organizados y entry point register.

PR	Merge en main	Commit	Alcance
#4	18a9fcc	ee36f24	Selección champion/challenger y comandos de reproducción del modelo elegido en el README.

Por una situación particular del equipo, el profesor autorizó que cada integrante desarrollara y entregara su propia versión del proyecto en un repositorio independiente. En consecuencia, esta entrega corresponde a la versión individual de Roberto Mendoza: los Pull Requests fueron abiertos e integrados por el autor y no tienen revisiones de otros integrantes. No se simularon revisiones ni colaboraciones, para mantener la trazabilidad real del trabajo.

7. Hallazgos de reproducibilidad y limitaciones

Durante la ejecución del plan se detectó que la ingesta reescribe la marca de tiempo `generado` de `data/manifest.json` aunque los datos no cambien. Como el manifiesto está versionado, el árbol de Git queda con cambios y las corridas posteriores se registran con `git_sucio = True`. Las corridas afectadas se repitieron con el árbol limpio y solo esas se usaron en el análisis y en el registro; el README indica restaurar el manifiesto después de la ingesta. La corrección definitiva, que la ingesta solo reescriba el manifiesto cuando cambie `data_version`, queda como mejora para la siguiente etapa.

Otras limitaciones a considerar: la duración de las corridas es una medida aproximada del costo de entrenamiento, porque depende de la carga del equipo (el Random Forest de P1 tardó 30 s en una ejecución y 150 s en otra); el desempeño de P2 en TEST está afectado por un cambio de prevalencia que conviene vigilar en producción, y en P3 la silhouette del modelo elegido es menor que la del baseline de dos grupos, lo que se aceptó a cambio de una segmentación más útil y más estable.

8. Conclusiones

La etapa convirtió el proyecto en un flujo reproducible y trazable de extremo a extremo: un MLflow Project ejecutable con un solo comando, corridas comparables bajo una convención de etiquetas común, búsquedas organizadas como nested runs, artefactos que respaldan cada resultado y un Model Registry en el que cada modelo en producción se puede rastrear hasta la corrida, el commit y la versión de datos que lo produjeron.

Los resultados confirman que el mejor modelo no es necesariamente el más complejo ni el de la mejor cifra aislada. En P1, un modelo lineal generalizado superó a los ensambles por

estabilidad y costo; en P2, el empate práctico entre dos configuraciones de gradient boosting se resolvió con un criterio fijado de antemano, y se dejó documentada la alternativa más barata; en P3, la elección combinó separación, estabilidad temporal y utilidad operativa de la segmentación. En los tres casos TEST se usó solo para confirmar la generalización, sin intervenir en la elección.

El esquema de aliases deja preparado el ciclo de vida para las siguientes etapas: los modelos `champion` son los que consumirán los servicios de inferencia, y cada mejora futura podrá registrarse, evaluarse como `challenger` y promoverse moviendo un alias, sin perder el historial ni el linaje de las versiones anteriores.

Referencias

Lakshmanan, V., Robinson, S., & Munn, M. (2020). *Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps*. O'Reilly Media.

MiBici. (s.f.). *Datos abiertos*. https://www.mibici.net/es/datos-abiertos/

MLflow. (s.f.-a). *MLflow dataset tracking* [Documentación]. https://mlflow.org/docs/latest/ml/dataset/

MLflow. (s.f.-b). *MLflow Model Registry* [Documentación]. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s.f.-c). *MLflow Model Registry workflows* [Documentación]. https://mlflow.org/docs/latest/ml/model-registry/workflow/

MLflow. (s.f.-d). *MLflow Projects* [Documentación]. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s.f.-e). *MLflow Tracking* [Documentación]. https://mlflow.org/docs/latest/ml/tracking/

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.